

Learning versus Planning for Orbit Wars: ctx2, a Hybrid Learned Re-Ranker, and NeRL, a Model-Predictive Planner

Vũ Nguyễn Đan, Phạm Tiến Dũng, Lê Hồng Anh

Group 12 — Team IAI-RL-DirtyDozen

Student IDs: 23020351, 23020345, 23020327

Submitting representative: Vũ Nguyễn Đan (23020351)

Course: Reinforcement Learning and Planning

University of Engineering and Technology – Vietnam National University, Hanoi (UET–VNU)
Hanoi, Vietnam

Abstract—This report studies two contrasting agent designs for the real-time strategy game *Orbit Wars*, both developed by our team. The first, *ctx2*, is a hybrid agent: a rule-based engine handles all physics and mechanics (candidate generation, fleet sizing, orbital lead-aiming, sun avoidance, combat resolution, defensive reinforcement), while the single highest-leverage strategic decision—which planet to attack and from where—is delegated to a compact neural network acting as a *re-ranker*. The network sees 230 features per candidate (46 base features plus DeepSet-style set statistics), uses an MLP $230 \rightarrow 128 \rightarrow 128 \rightarrow 1$, is trained by supervised regression on roughly 510 000 labelled candidates, and is exported to pure Python to run inside the one-second-per-turn budget. On a held-out split it matches the top-ranked label about 81–85% of the time; its public score is about 1025 (current entry 1024.7; it converged near 1067 earlier).

The second agent, NeRL (a backronym for *Not even RL*), is a model-predictive planner. It carries *no* learned parameters: it builds a tensorised forward model of the game, projects each planet’s future ownership and garrison over a horizon, and selects launches by their *marginal effect* on that projected state (a competitive ship-flow objective), followed by greedy multi-wave selection, capture-floor ship economy, coordinated multi-source strikes (*focus-fire*), and defensive regrouping. In head-to-head play this planner eliminates both *ctx2* and our strongest heuristic outright; as our active leaderboard submission it currently rates 1154.6 (TrueSkill, still settling), above the re-ranker. We discuss why an exact-model planner overcomes the imitation ceiling and distribution shift that fundamentally limit the supervised re-ranker, and what this implies for combining learning with planning.

Index Terms—Orbit Wars, model-predictive planning, receding horizon, supervised learning, re-ranker, DeepSet, game AI, self-play

I. TEAM, CONTRIBUTIONS, AND REPOSITORY

A. Group and members

This work is submitted by **Group 12** on the class list, competing under the team name IAI-RL-DirtyDozen. The members and student IDs are:

- Vũ Nguyễn Đan (23020351) — *submitting representative*
- Phạm Tiến Dũng (23020345)
- Lê Hồng Anh (23020327)

B. Work division and contributions

During the first week (25/05–31/05) each member worked on a separate branch and independently explored a different direction. From 01/06 to 09/06 the team consolidated onto the strongest line: we took as the base the work of whichever member held the highest leaderboard score and developed from there, adjusting flexibly as scores changed. The resulting roles and contributions are summarised in Table I.

Table I
MEMBER ROLES AND CONTRIBUTIONS

Member	Role and contribution	Share
Vũ Nguyễn Đan	Team lead; <i>ctx2</i> agent; report writing	40%
Phạm Tiến Dũng	<i>NeRL</i> agent; report writing	40%
Lê Hồng Anh	Active in the first week; not active in the final week	20%

C. Repository

All code—both agents, the training pipeline, the evaluation harness, and the source of this report—is available at:

<https://github.com/dxpawn/OrbitWars>

D. Leaderboard standing (summary)

At the time of writing, IAI-RL-DirtyDozen sits at public score **1154.6**, global rank **575 / 4016** teams, and **11th of the 17** IAI-RL-* class teams. The full class breakdown is given in Sec. VI-C, Table V.

II. INTRODUCTION

Orbit Wars is a two-dimensional real-time strategy problem: planets orbit a central sun, each produces resources and garrisons ships, and players launch fleets to capture planets. Every turn an agent must decide how many ships to send, from which planet, to which target, balancing expansion, defence, and offence. The action space is large and the orbital

dynamics are non-trivial, so exhaustive global search within the competition’s per-turn time budget is infeasible.

We approached the problem from two different directions, and this report presents both as a single comparative study.

Approach A — learning where it matters (*ctx2*). We separate *mechanics* from *judgement*. Everything that can be written down and tested as a rule—fleet sizing, orbital aiming, sun avoidance, combat, reinforcement—is handled by a rule engine. Only the high-value judgement call, “which of many feasible targets is worth attacking,” is learned from self-collected data. This reflects a familiar principle: do not use machine learning where a reliable rule already exists; concentrate model capacity on the highest-value bottleneck.

Approach B — planning with an exact model. Rather than *learning* a target preference, the planner *computes* one. Because the game’s transition rules are known, we simulate the near future exactly and choose the launches that move the projected outcome most in our favour. This is model-based planning in the receding-horizon / model-predictive-control sense [4], [5]: the model is *given*, not learned.

Contributions.

- 1) A hybrid engine-plus-re-ranker agent (*ctx2*) with a DeepSet-style permutation-invariant set encoding, trained by offline supervised learning and exported to dependency-free Python (Sec. IV).
- 2) A tensorised model-predictive planner that projects garrison ownership forward and scores launches by a marginal competitive ship-flow objective, with capture-floor ship economy, focus-fire, and regrouping (Sec. V).
- 3) An honest empirical comparison of the two paradigms, including the imitation ceiling, distribution shift, and the unreliability of local self-play as a ranking signal (Sec. VII).

III. PROBLEM AND GAME MECHANICS

The playing field is 100×100 with a sun of radius $R_{\text{sun}} = 10$ at the centre. Planets may be static or move on orbits; the maximum fleet speed is 6 units per turn and a match lasts up to 500 turns. Each turn the agent receives an *observation* describing every planet (owner, ship count, production, position), all in-flight fleets, and incoming threats. Fleet speed depends on fleet size; a fleet aimed at a moving planet must lead its target; and fleets are destroyed if their path crosses the sun. Capturing a planet requires arriving with more ships than the defender holds at the moment of arrival, after which production accrues to the new owner.

These mechanics are fully known and deterministic given the players’ actions. That single fact is what makes Approach B possible: a known transition function can be *simulated* rather than *learned*.

IV. APPROACH A: HYBRID ENGINE AND LEARNED RE-RANKER (CTX2)

A. Scope of the learned component

ctx2 does **not** learn an end-to-end policy. The network participates only at the *re-ranking* step: after the engine enumer-

ates feasible attack candidates for each owned source planet, the network assigns a quality score and adjusts the priority order. Everything else—computing the fleet size needed to win and hold, leading a moving target, avoiding the sun’s shadow, counter-snipe handling, reinforcing threatened planets—is done by the rule engine in `engine/orbit_base.py`.

B. Per-turn decision pipeline

The pipeline has six steps:

- 1) **Parse observation.** Decode the state into planets (owner, ships, production, orbital position), in-flight fleets, per-side totals, and incoming threats.
- 2) **Candidate generation.** For each owned planet, the engine lists targets capturable within movement range, pre-sorted by a weighted-distance heuristic. Each candidate’s position in the merged list is recorded as the index `idx`. An expansion factor `expand = 10` admits extra candidates before truncating to the top- K .
- 3) **Featurisation.** Each (source, target) pair is encoded as a 46-dimensional vector (Sec. IV-E).
- 4) **Neural scoring.** `score_many(rows)` receives all candidates of a call, appends set statistics (mean, max, min, std per dimension), normalises, and runs the MLP forward pass.
- 5) **Re-ranking.** The final priority is

$$\text{priority} = \text{idx} - \text{bonus} \cdot \sigma(\text{score}), \quad (1)$$

with σ the sigmoid and `bonus = 1.45 (2p) or 1.25 (4p)`. A high learned score pulls a candidate above its heuristic order; `bonus` caps the network’s maximum influence. In 4p a tiny jitter (10^{-6}) breaks near-ties.

- 6) **Execution.** The engine sends fleets to the chosen targets: it computes the ships required to win and hold, leads the moving target, avoids the sun, and runs a defensive reinforcement pass.

Typical network inference is about **0.1 s/turn**, comfortably inside the one-second budget.

C. The rule engine

The engine in `orbit_base.py` carries most of the rule-based combat intelligence:

- **Fleet sizing and navigation:** speed as a function of fleet size, predicting the target’s position at arrival (lead aiming), sun-collision checks.
- **Initial target heuristic:** weighted distance, a preference for static planets, an expansion bonus, and race-condition handling when several sides target the same planet.
- **Short-horizon forward projection (4p):** a 7-turn look-ahead that adjusts candidate ranking for longer-than-one-step value.
- **Active defence:** snipe / counter-snipe detection, reinforcing weak planets, late recapture in 2p.
- **Multi-action orchestration:** a per-turn action cap (7 in 2p, 5 in 4p) and source-expansion search.

Separating engine from network aids debugging: mechanics and candidate generation can be tested independently, enabling the re-ranker only when evaluating the learned part.

D. Network architecture

The re-ranker is a three-layer MLP with ReLU activations, trained by supervised regression to predict a target-quality score from the feature vector. After training, the weights are exported to pure Python—no PyTorch/TensorFlow dependency at competition time.

a) *230-dimensional input and DeepSet encoding*: Each candidate i within a decision has a base vector $\mathbf{x}_i \in \mathbb{R}^{46}$. Before the MLP, we append set statistics over all candidates S of that decision:

$$\boldsymbol{\mu} = \text{mean}_{j \in S}(\mathbf{x}_j), \quad \mathbf{m} = \max_{j \in S}(\mathbf{x}_j), \quad (2)$$

$$\mathbf{n} = \min_{j \in S}(\mathbf{x}_j), \quad \boldsymbol{\sigma} = \text{std}_{j \in S}(\mathbf{x}_j), \quad (3)$$

(all per-dimension). The full input is $\mathbf{z}_i = [\mathbf{x}_i; \boldsymbol{\mu}; \mathbf{m}; \mathbf{n}; \boldsymbol{\sigma}] \in \mathbb{R}^{230}$.

This is a **DeepSet-style set-pooling** [1]: the network judges a candidate *relative* to the other options in the same decision, rather than in isolation, while remaining cheap pointwise inference. The model is **permutation-invariant** over candidate order and handles a variable number of candidates per turn.

b) *Forward pass*: After normalisation $\hat{\mathbf{z}} = (\mathbf{z} - \boldsymbol{\mu}_{\text{train}})/\boldsymbol{\sigma}_{\text{train}}$ (the MEAN, STD constants are stored in the weight file):

$$h_1 = \text{ReLU}(W_0 \hat{\mathbf{z}} + b_0), \quad h_2 = \text{ReLU}(W_1 h_1 + b_1), \quad s = W_2 h_2 + b_2. \quad (4)$$

The architecture is $230 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{linear}} 1$. The scalar s , after a sigmoid, enters Eq. (1). Two separate weight sets (`student_weights_2p.py`, `student_weights_4p.py`) are kept because the state distribution and re-rank hyperparameters differ between 2p and 4p.

E. The 46 base features

Features are extracted in `engine/main.py`, combining global state (shared by all candidates in a turn) with local (source, target) information. Most values are clamped to $[0, 1]$ or $[-1, 1]$; ship counts pass through a saturating map $x/(x + 40)$ to reduce sensitivity to absolute scale. Table II lists all 46 dimensions in vector order.

A short-horizon history layer (`_FeatureHistory`) keeps a 5–10 turn buffer to compute *momentum*, *aggression_trend*, *enemy_rhythm*, *approach_rate*, and *convergence_threat*—trend features that let the network sense game phase and opponent pressure rather than a single snapshot. The re-rank hyperparameters live in `engine/feature46_manifest.json`: `bonus_2p=1.45`, `bonus_4p=1.25`, `expand=10`.

F. Training method

The model is trained by **offline supervised learning**: collect (features $\rightarrow$ quality-score) pairs, then regress. There is no environment reward and no exploration—only MSE regression onto fixed labels.

Quality labels. For each decision, an internal scoring pipeline ranks every candidate from its 46 features and set context. This score expresses the strategic priority assigned by the reference scorer and serves as the regression target for the lighter MLP.

Data collection.

- Run the agent over **≈ 250 games** (2p and 4p) against a diverse pool (`heuristic_v6`, `adv_hellburner`, `adv_proto_v15`, `adv_lb958`).
- Each `score_many` call is tagged with a *group id*; all candidates of a call share a group, so set context can be recomputed at training time.
- Size: $\approx 25\,000$ decision groups (2p), $\approx 37\,000$ (4p); **$\approx 510\,000$** labelled candidates total.

Training.

- Concatenate the 46 features with the group’s mean/max/min/std $\Rightarrow$ 230 dims; normalise by global mean/std.
- Split 90/10 *by group* (not by row) to avoid leakage between train and validation.
- MSE loss between predicted and target score; Adam on GPU; about **1 minute** of training.
- Primary metric: *held-out group top-1 agreement*—within each validation group, does the model’s top candidate match the label?

Export. Weights, MEAN, STD are written to `model/student_weights_{2p,4p}.py`; `score_many` reproduces the set-pooling and forward pass exactly as in training.

V. APPROACH B: NeRL, A MODEL-PREDICTIVE PLANNER

NeRL—a backronym for *Not even RL*, since it uses no learning at all—shares the game with *ctx2* but inverts the philosophy: instead of *learning* which target to prefer, it *simulates* the consequences of every candidate launch and picks the one that most improves the projected outcome. The name is deliberately self-aware: NeRL sits on the *planning* side of the course (model-based planning with a known model) rather than the learning side. It contains **no neural network and no learned weights**; torch is used purely as a fast vectorised-math library so the whole simulation fits in the time budget.

A. What the agent is, in RL terms

Because the transition rules are known exactly, the planner is **model-based planning with a given model** [5]—a receding-horizon / model-predictive controller [4]. It is *not* a reactive heuristic (it does not score the present state; it simulates the future), and it is *not* learned. The only heuristic elements are pragmatic approximations that keep planning tractable: candidate shortlisting, greedy (rather than optimal) selection, a fixed horizon, and the assumption that opponents launch nothing new beyond fleets already in flight (no adversarial game-tree search).

Table II
THE 46 BASE FEATURES (ORDER AS IN `_CANDIDATE_FEATURES`)

#	Name	Short description
<i>Global / macro state (indices 0–11)</i>		
0	game_progress	Current turn / 500
1	players_alive	Number of active players / 4
2	my_planet_share	Fraction of planets we own
3	my_gdp_share	Fraction of total production we own
4	my_ship_share	Fraction of all ships (planet + transit) we own
5	momentum	Change in our ship share vs. 5 turns ago
6	my_fleet_ratio	Ships in transit / our total ships
7	leader_gap	Normalised force gap to the leading side
8	am_i_targeted	Fraction of enemy fleets aimed at our planets
9	fastest_grower_gap	Momentum gap between the fastest-growing rival and us
10	aggression_trend	Trend of being targeted (5 turns)
11	enemy_rhythm	Variability of enemy attack tempo (10 turns)
<i>Source planet and diplomacy (12–19)</i>		
12	src_ships	Source ship count (saturating)
13	src_production	Source production / 5
14	src_safety	Source safety against incoming threats
15	diplomacy	+1 our defence, 0 neutral, –1 attacking a rival
16	target_production	Target production / 5
17	is_dynamic_target	1 if comet or non-static planet
18	owner_power	Target owner’s ship share of the board
19	is_weakest_enemy	1 if target belongs to the weakest rival
<i>Capture and threat simulation (20–31)</i>		
20	proj_owner	Projected owner at arrival: us / neutral / enemy
21	proj_ships	Projected defender ships at arrival (saturating)
22	threat_after	Enemy threat arriving <i>after</i> we capture
23	support_after	Our support arriving after we capture
24	pre_volatility	Force volatility before we arrive
25	threat_potential	Attack potential from other enemy planets
26	support_potential	Support potential from our other planets
27	eta	Time to arrival / 25
28	send_fraction	Ships sent / our total ships
29	need_ratio	Ships needed to win / source ships
30	margin	Safety margin (send – need), normalised
31	can_win	1 if we send enough ships to capture
<i>Strategy and terrain (32–45)</i>		
32	garrison_strength	Remaining force after capture (saturating)
33	defense_sustainability	Ability to hold after capture
34	target_roi	Target production / (ships needed +1)
35	front_diff	Distance-to-nearest-enemy gap (source vs. target)
36	avg_enemy_distance	Mean distance to the enemy centroid
37	angular_spread	How much enemies surround us angularly
38	orbital_trend	ETA trend due to orbital motion
39	convergence_threat	Convergence threat (enemies closing distance)
40	approach_rate	Rate enemies approach our centroid
41	enemy_commitment	Enemy fraction of ships in transit
42	weakest_colony	Our weakest planet (after subtracting incoming threats)
43	local_superiority	Local support-vs-threat advantage
44	economic_impact	Target production / our production
45	enemy_exposed	How exposed the enemy side is

B. Per-turn pipeline

Algorithm in `run_turn`:

- 1) **Parse** the observation into tensors (planets, in-flight fleets).
- 2) **Build/refresh the movement model.** `PlanetMovement` predicts every planet’s position over the horizon H , tracks in-flight fleets through a ledger, and supports forward projection of garrisons.
- 3) **Distance cache.** Cross-time distances “planet s at $t=0$ to planet t at step k ”—the geometrically correct quantity for intercept feasibility against moving planets.
- 4) **Garrison projection.** `garrison_status` returns, for steps $0 \dots H$, each planet’s projected (owner, ships) via an *exact* recurrence: production, fleet arrivals, and combat resolved step by step.
- 5) **Plan waves** (Sec. V-C).
- 6) **Emit** the chosen launches as a sparse action payload.

C. Wave planning and scoring

a) *Shortlists*: Sources are owned planets with at least `min_ships_to_launch` ships, top-ranked by ship count. Targets combine an *offensive* shortlist (enemy/neutral planets by proximity) and a *defensive* shortlist of *flip targets*—our own planets projected to be lost within the horizon, ranked by $\text{urgency} = \text{prod} \cdot (\text{turns left}) + \text{ships}$.

b) *Fleet sizing and reachability*: For each (source, target), the launch size is `safe_drain`: the most ships the source can send while still holding itself over the horizon. An analytic reachability test (accounting for orbital drift, sun line-of-sight, and surface offsets) and an `intercept_angle` solver (fixed-point iteration) give the aim angle and ETA to the moving target.

c) *Capture floor*: `capture_floor` computes the *minimum* ships to take a target at its arrival turn:

$$f_k = \lceil \text{defenders}_k + \text{overhead} + \text{reinforcements}_k \rceil, \quad (5)$$

where defenders_k is the projected garrison at arrival step k . A candidate is kept only if its size clears f_k . This yields disciplined ship economy: the agent never over-commits to a capture.

d) *Focus-fire*: When no single source clears a target’s floor, the planner *pools* multiple sources whose fleets arrive on the *same* turn and whose cumulative ships exceed f_k , forming a coordinated multi-source strike. (This is the main capability the strongest configuration adds over a single-source planner.)

e) *Marginal competitive scoring*: Each candidate launch set c is scored by its effect on the projected ship flow. Let $\Delta\eta_j(c)$ be the change in player j ’s projected *net* ships (produced minus lost in combat over the horizon) induced by c . The baseline objective is

$$s(c) = \Delta\eta_{\text{me}}(c) - \sum_{j \neq \text{me}} \Delta\eta_j(c). \quad (6)$$

This is a one-ply look-ahead over the exact simulator: it rewards launches that increase our net ships while reducing opponents’ net ships.

f) *Greedy selection and regroup*: A greedy loop selects the highest-scoring non-conflicting waves up to `max_waves_per_turn`, respecting per-source budgets and an ROI threshold (fire only if $s(c) > \text{roi_threshold}$). Finally, a *regroup* pass moves leftover ships from safe planets toward threatened friendly planets, following an enemy-pressure gradient that includes in-flight enemy fleets.

D. Configurations

A frozen `ProducerLiteConfig` holds the knobs. The 2p preset uses `horizon = 18`, up to 6 waves/turn, `ROI = 1.5`, focus-fire on (up to 4 strike sources). The 4p preset shortens the horizon to 13 and reduces sources, defensive targets, and strike sources, reflecting the higher branching and shorter useful look-ahead in a four-way game.

VI. EXPERIMENTAL RESULTS

A. *ctx2*: held-out label agreement

Table III
CTX2 TRAINING METRICS ON THE HELD-OUT (BY-GROUP) SPLIT

Metric	2p	4p
Group top-1 agreement	81.3%	84.8%
R^2 (MSE on scores)	0.917	0.933
Aggregate: top-1 $\approx 85\%$, top-3 $\approx 95\%$, $R^2 \approx 0.93$		

Top-1 agreement $\approx 85\%$ means that in about 85% of re-rank decisions the model’s top candidate matches the label; top-3 $\approx 95\%$ indicates the highest-labelled target is usually within the model’s top three—errors are mostly secondary ordering, not missing the key target. The public score is about **1024.7** (it converged near 1067 earlier; TrueSkill drift ± 30 on the board). Inference is ≈ 0.1 s/turn and the submission needs only standard Python at competition time.

B. NeRL dominates head-to-head

In direct play (our local harness, built on `kaggle_environments`) NeRL is decisively stronger than both our heuristic and *ctx2*:

Table IV
NeRL VS. OUR OTHER AGENTS (2P, REPRESENTATIVE SEEDS)

Match (NeRL = P0)	Winner	Score	Turns to win
NeRL vs. <i>heuristic_v6</i>	NeRL	1211–0	85 / 500
NeRL vs. <i>ctx2</i>	NeRL	1478–0	94 / 500

In both 2p matches NeRL eliminates the opponent before turn 100 of 500. As our active leaderboard submission it currently rates **1154.6** (TrueSkill, still converging)—above *ctx2*’s 1024.7 and consistent with the head-to-head dominance, though only mid-pack against the full 4016-team field (Sec. VI-C). The local domination of *ctx2*/v6 and the global rating measure different things: the former is a direct duel, the latter is a rating against the entire population. Per-turn

compute (vectorised torch on CPU) stays well under the one-second budget despite performing a full forward projection every turn.

C. Leaderboard standing

Table V places our team within the IAI-RL-* class cohort on the public Orbit Wars leaderboard (snapshot of 08/06; TrueSkill ratings drift by ± 30 and keep updating as submissions continue to play). Our active submission is **NeRL** at public score **1154.6**; our *ctx2* submission scored **1024.7**. Among 4016 teams overall we rank **575th**; among the 17 class teams we rank **11th**. We report this plainly: NeRL dominates our *own* earlier agents head-to-head, but on the global ladder it is mid-field, and several class teams running similar strong public agents rate above us. The value of this project is therefore less the raw number and more the comparative understanding of *why* planning outperforms our learned re-ranker (Sec. VII).

Table V
IAI-RL-* CLASS COHORT ON THE PUBLIC LEADERBOARD (08/06 SNAPSHOT,
4016 TEAMS TOTAL)

Class #	Team	Score	Global rank
1	IAI-RL-06	1314.9	61
2	IAI-RL-IX	1259.4	114
3	IAI-RL-II	1228.3	191
4	IAI-RL-NapNap	1222.5	209
5	IAI-RL-17	1212.7	254
6	IAI-RL-GaNhatVienAI	1202.0	337
7	IAI-RL-Tlm71	1201.2	343
8	IAI-RL-Beginner	1194.6	393
9	IAI-RL-Checkm8	1186.3	439
10	IAI-RL-04	1176.0	496
11	IAI-RL-DirtyDozen (ours)	1154.6	575
12	IAI-RL-03	1115.0	640
13	IAI-RL-codex	1108.8	645
14	IAI-RL-RUN	1053.3	693
15	IAI-RL-10	935.7	941
16	IAI-RL-Cherry_11	887.1	1033
17	IAI-RL-Trauma07	696.6	1957

VII. DISCUSSION: LEARNING VS. PLANNING

A. The ceiling of offline supervised learning

ctx2's objective is to *match a label*, not to *maximise win rate*. Our experiments confirmed that higher label agreement does not always mean stronger play—a variant that fit the labels more tightly actually scored *lower*—so **label agreement $\neq$ competitive strength**. A supervised re-ranker can, at best, approach the quality of the scorer that produced its labels; it cannot exceed it. This is the fundamental ceiling that the planner sidesteps: by optimising a *consequence* (projected ship flow) rather than imitating a *preference*, the planner is not bounded by any teacher.

B. Why the planner wins

The planner has three structural advantages over the re-ranker:

- 1) **It optimises outcomes, not labels.** Eq. (6) is grounded in the actual game dynamics; the re-ranker is grounded in a fixed scoring heuristic.
- 2) **Exact ship economy.** The capture floor sends the minimum winning fleet, freeing ships for more targets—an efficiency the feature-based re-ranker only approximates.
- 3) **Coordination.** Focus-fire executes multi-planet strikes that a per-candidate re-ranker, scoring each launch independently, cannot represent.

C. Distribution shift and compounding errors

At training time *ctx2*'s data lies on the state distribution induced by the collection pipeline; at competition time the agent follows its *own* distribution, and the mismatch causes **compounding errors**. The principled remedy is **Dagger** [2]: relabel the states the agent actually visits and retrain. The planner is immune to this failure mode—it re-plans from the true state every turn, so there is no train/test distribution gap.

D. Reliability of evaluation

Local self-play and win rate against a fixed pool do *not* correlate tightly with the public leaderboard, because of distribution shift to the real competitor population and TrueSkill drift. We therefore trust only **paired held-out metrics** (agreement, R^2) and **direct leaderboard results**, and we treat any cross-day absolute-score comparison with suspicion.

E. Limits and future work

The planner's largest simplification is the absence of opponent modelling: it assumes rivals launch nothing new beyond fleets already in flight. Natural extensions toward opponent awareness are (i) a shallow adversarial roll-out (simulate the strongest rival's likely best reply), (ii) replacing greedy wave selection with a small beam search, and (iii) using *ctx2*'s learned score as a *tie-breaker* inside the planner's shortlist—a concrete way to combine the two paradigms, with learning proposing and planning disposing.

VIII. IMPLEMENTATION STRUCTURE

Table VI shows the packaged model bundle; the full development repository—both agents, the training pipeline, and the evaluation harness—is at the GitHub link in Sec. I.

Reproduction (Approach A): collect $\rightarrow$ train $\rightarrow$ validate $\rightarrow$ build submission; the ≈ 85 MB .npz data can be regenerated by the collection script. Approach B has no training step—the agent is self-contained and deterministic given the seed.

IX. CONCLUSION

We presented two agents for Orbit Wars and an honest comparison of their paradigms. *ctx2* is a hybrid design—a rule engine for mechanics and a compact DeepSet re-ranker (230-128-128-1) for target selection—trained on $\approx 510\,000$ candidates to ≈ 81 – 85% top-1 agreement and $R^2 \approx 0.92$ – 0.93 , scoring about 1024.7. NeRL, our model-predictive planner, carries no learned parameters yet rates 1154.6 on the public leaderboard—above *ctx2*—and eliminates both *ctx2* and

Table VI
PACKAGED SUBMISSION LAYOUT (CTX2 MODEL BUNDLE); NERL SHIPS AS ONE
STANDALONE FILE

Path	Role
<i>Approach A — ctx2 (learned re-ranker)</i>	
agent/submission_distilled_ctx2.py	Single-file competition submission
model/student_weights_{2p,4p}.py	MLP weights + score_many
engine/orbit_base.py	Rule engine (gameplay)
engine/main.py	Candidate generation, 46 features, re-rank
engine/feature46_manifest.json	Feature schema + re-rank hyper-params
training/distill_collect_grouped.py	Grouped data collection
training/distill_train_ctx.py	Train + export
training/validate_student.py	Held-out agreement check
training/selfplay.py	A/B self-play
<i>Approach B — NeRL (model-predictive planner)</i>	
agents/NeRL.py	NeRL planner, single file

our heuristic head-to-head, because it optimises the simulated *outcome* rather than imitating a fixed preference. The broader lesson is the classic learning-versus-planning trade-off: when an exact model is available, planning against it can outperform learning to imitate, and the most promising direction is to let learning *propose* and planning *dispose*.

ACKNOWLEDGEMENT

We thank the course instructors and the Orbit Wars organisers for the competition environment, benchmarks, and guidance, and the competitor community for real-world feedback on agent performance.

REFERENCES

- [1] M. Zaheer *et al.*, “Deep sets,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” *AISTATS*, pp. 627–635, 2011.
- [3] A. Hussein *et al.*, “Imitation learning: A survey of learning methods,” *ACM Computing Surveys*, vol. 50, no. 2, pp. 1–35, 2017.
- [4] C. E. Garcia, D. M. Prett, and M. Morari, “Model predictive control: Theory and practice—a survey,” *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.